

Guia de decisão: disponibilidade e desempenho

Base para failover, balanceador e réplica — ERP Nascimento · setembro/2026

Para que serve este documento

Consolidar o que foi **medido em produção** durante os incidentes de 21 e 23 de setembro de 2026, e servir de régua na hora de decidir onde investir: réplica, failover, balanceador, cache ou máquina maior.

A ideia é evitar o erro mais caro de infraestrutura: **comprar a ferramenta certa para o problema errado**.

A pergunta que originou este guia

"Não quero um sistema lento que vai ficando cada vez mais lento conforme aumentam os eventos. Sei que é melhor lento e funcionando do que rápido e caindo — mas se der para ter os dois, melhor."

Dá para ter os dois. A seção 3 explica por quê: **velocidade e confiabilidade não são o mesmo eixo**.

1. O que realmente aconteceu — três quedas, três causas

DATA	MÁQUINA	O QUE ESTOUROU	PLATAFORMA SUPABASE
31/08/2026	Nano · 0,5 GB	CPU esgotada	operacional
21/09/2026	Micro · 1 GB	Conexões: 61 em uso para 57	operacional
23/09/2026	Micro · 1 GB	Memória: swap, commit 2,46 GB em 1 GB	operacional

O fato que decide todo o resto

Nas três, o painel de status da Supabase estava **operacional**. Nenhuma queda veio do fornecedor. As três foram a mesma máquina pequena batendo em três paredes diferentes. Em nenhuma delas o banco *morreu* — ele ficou **lento demais para servir**.

O que resolveu

Upgrade de **Micro para Small** em 23/09 às 17:36. Medido por dentro do banco:

	MICRO	SMALL
Conexões utilizáveis	57	87
<code>shared_buffers</code>	256 MB	512 MB
<code>effective_cache_size</code>	768 MB	1,5 GB
<code>work_mem</code>	3,5 MB	5 MB

Custo: ~US\$ 5/mês. A parada do upgrade durou **30 segundos**. Nas 3 horas seguintes: 360 medições, zero quedas.

Por que a máquina era pequena demais — a conta que não fechava

No Micro: `shared_buffers` 256 MB + `work_mem` 3,5 MB × 60 conexões (205 MB) + `maintenance_work_mem` 64 MB = **525 MB antes de qualquer conexão existir**. Somando ~5–10 MB por conexão aberta e os processos de PostgREST, Auth e Storage *na mesma máquina*, **a configuração padrão já era maior que a máquina**. Por isso o swap.

2. A régua: qual ferramenta resolve qual problema

Este é o diagrama mais importante do documento. Antes de escolher uma solução, identifique **em qual coluna** está o sintoma.



Figura 1 — Sintoma → categoria → ferramenta. Ferramenta de uma coluna não resolve problema de outra.

Aplicando aos três incidentes

As três quedas caem na coluna **CAPACIDADE**. Réplica, failover e balanceador vivem na coluna **REDUNDÂNCIA** — e não teriam evitado nenhuma delas.

3. "Rápido" e "confiável" não são o mesmo eixo

A intuição comum é que garantia custa velocidade — que um sistema seguro é necessariamente lento. **Na maior parte dos casos, não é verdade.**

A lentidão que o ERP tinha **não vinha de nenhuma garantia**. Vinha de trabalho desnecessário. Exemplos medidos e corrigidos em 21/09, sem afrouxar nada:

DESPERDÍCIO	ANTES	DEPOIS
Cache de consultas (<code>staleTime</code> ausente)	~404 consultas refeitas à toa	30 s de cache
Consulta por linha de lista (N+1)	~13 consultas/segundo	2 por lista
Cards desenhados de uma vez	2.105	24 por vez
Busca por digitação	1 consulta por tecla	1 por palavra
Setores de EMPREGADOS	13.279 linhas	14 linhas

A regra para ter os dois

Velocidade vem de fazer menos trabalho, não de garantir menos. Cache, paginação, consulta em lote e índice deixam o sistema rápido sem abrir mão de nada.

A garantia forte — commit síncrono — se aplica **só onde há dinheiro**, que são poucas operações por dia. O resto do sistema nem sente. É isso que permite ser rápido e confiável ao mesmo tempo.

Sobre "ficar mais lento conforme os eventos aumentam"

Esse medo é legítimo e tem nome: **crescimento não linear**. Acontece quando o trabalho cresce mais rápido que o dado — por exemplo, uma tela que faz uma consulta por linha: dobrar os registros dobra as consultas. Era exatamente o caso do Malote.

O antídoto não é máquina maior. É **garantir que cada operação custe o mesmo independente do volume**: paginar, consultar em lote, indexar. Máquina maior só adia o problema.

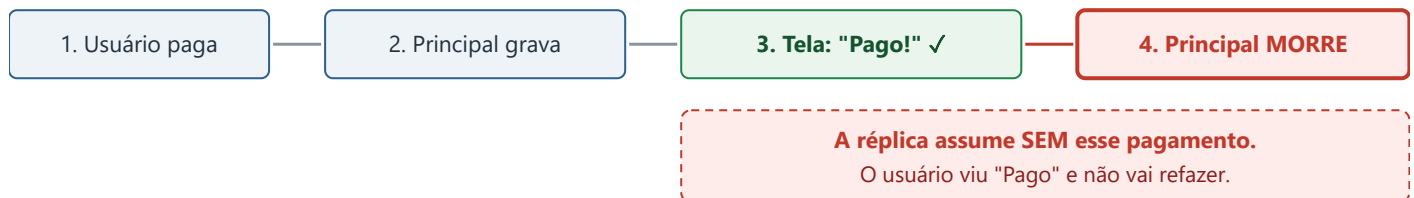
4. Replicação: síncrona x assíncrona

A decisão mais importante ao montar réplica — e a mais mal explicada.

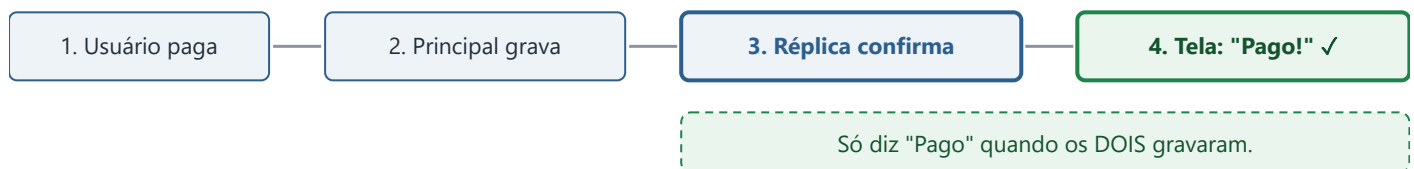
Desfazendo o mal-entendido mais comum

Nenhum dos dois modos grava dado incompleto. Isso é garantido pelo ACID do Postgres, com ou sem réplica. "Meio pagamento gravado" não existe em nenhum cenário. A diferença entre os modos é **o que a réplica tem no momento do failover**.

ASSÍNCRONO — a janela de perda



SÍNCRONO — sem janela



O meio-termo: escolher por transação

Dentro das RPCs de dinheiro (malote_pagar_despesa, malote_pagar_parcela, malote_aprovar_despesa):

```
SET LOCAL synchronous_commit = 'remote_apply';
```

← só o pagamento espera.

Figura 2 — A janela de perda do assíncrono, e o meio-termo por transação.

O risco real do assíncrono

Não é "não gravou". É **gravou, o sistema confirmou ao usuário, e sumiu**. O usuário viu "Pagamento realizado" e não vai refazer — o problema aparece na conciliação, dias depois. Para dinheiro, isso é pior que um erro na tela.

O que quase ninguém conta sobre o síncrono

Síncrono com UMA réplica reduz a disponibilidade. O principal só confirma depois que a réplica grava; se a réplica cair ou ficar lenta, **o principal para de aceitar escrita**. Você adicionou uma máquina para ter mais disponibilidade e passou a depender de duas. Para ter segurança e disponibilidade são necessárias **duas réplicas com quórum** — três bancos no total.

A recomendação: escolher por transação

O Postgres permite decidir por operação, não globalmente. Como os caminhos de dinheiro do ERP já estão isolados em RPC, o ponto de aplicação já existe:

```
-- dentro de malote_pagar_despesa, malote_pagar_parcela, malote_aprovar_despesa
SET LOCAL synchronous_commit = 'remote_apply';
```

OPERAÇÃO

MODO

EFEITO

Pagamento, aprovação	síncrono	zero perda; poucas por dia, latência irrelevante
Todo o resto	assíncrono	rápido, como hoje

5. Failover e alta disponibilidade

O que HA protege — e o que não

Protege contra o banco MORRER: máquina, disco, zona, processo derrubado.

NÃO protege contra o banco ficar LENTO. E há duas agravantes: (1) saturação muitas vezes **não dispara o failover**, porque o health check ainda responde; (2) se disparasse, a réplica é do **mesmo tamanho** — receberia a mesma carga e saturaria também.

Split-brain: o desastre que HA mal feito cria

Se o failover erra e **os dois bancos aceitam escrita**, os dados divergem.

CENÁRIO	CONSEQUÊNCIA
Fora do ar	Ninguém grava. Reinicia e tudo volta consistente. <i>Foi o que aconteceu.</i>
Split-brain	Dois registros do mesmo pagamento em bancos diferentes. Alguém decide na mão qual vale.

Para um sistema com dinheiro, **o segundo é muito pior**. HA mal configurado é mais perigoso que não ter HA.

Checklist antes de ligar failover automático

- Controlador de failover definido (Patroni, repmgr, pg_auto_failover ou HA gerenciado)
- **Quórum** para decidir a promoção — evita split-brain
- Mecanismo de **fencing**: garantir que o antigo principal não volta aceitando escrita
- Redirecionamento de conexão resolvido (proxy, IP virtual ou DNS com TTL baixo)
- `synchronous_commit` definido **por transação** (seção 4)
- Failover **ensaiado** em homologação antes de existir em produção
- Runbook de **volta**: como o antigo principal reentra sem sobrescrever dados

6. Balanceador de carga

Distribui requisições entre **várias cópias do serviço HTTP**. Resolve dois problemas: uma instância morrer e uma instância não dar conta sozinha.

A armadilha do balanceador com banco único

Balanceador multiplica **quem faz pedido**, não **quem responde**. Se o gargalo é o banco, adicionar réplicas HTTP **piora**: cada nova instância abre seu próprio pool de conexões contra o mesmo Postgres. Em 21/09 o teto de conexões já foi o que derrubou o sistema — e isso com **uma** instância.

É o risco que a própria issue de DevOps registra: "*Escalar pods sem controlar conexões pode saturar o banco.*" Essa previsão se confirmou em produção no dia seguinte à sua publicação.

Regra: antes de escalar horizontalmente, dimensionar `max_connections` e o pooling. Cada réplica precisa de um teto de conexões, e a soma dos tetos tem que caber no banco.

7. A escada de disponibilidade

Cada degrau compra uma coisa diferente. O erro comum é achar que backup dá continuidade — **backup protege o dado, não o serviço**.

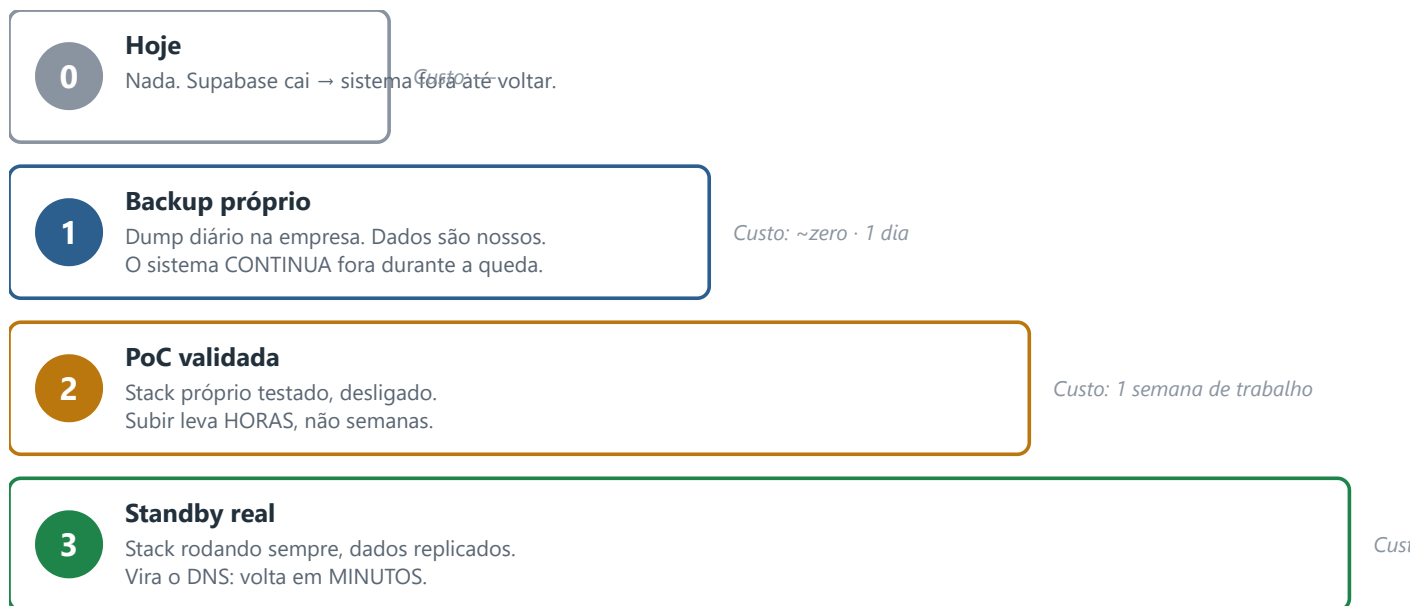


Figura 3 — O que cada nível entrega se o banco principal cair.

Como responder "o que temos se a Supabase cair?"

Hoje: nada. Esperamos voltar — foi o que aconteceu em 23/09, por 20 minutos. O Nível 1 muda isso para "os dados são nossos"; só o **Nível 3** significa "o sistema continua no ar".

A parte desconfortável

Um standby operado por **uma pessoa** tende a ter recuperação **pior** que a plataforma gerenciada. Comparação real: em 21/09 a Supabase reiniciou sozinha (~1 min fora); em 23/09 não reiniciou, e a recuperação dependeu de alguém perceber e clicar (~20 min). Failover manual é limitado pelo tempo humano, não pelo técnico.

8. Ordem recomendada de ações

#	AÇÃO	RESOLVE	CUSTO
1	Alerta de saúde — testar a API com e sem chave a cada minuto; avisar por WhatsApp em 2 falhas seguidas	Saber antes do usuário	meio dia
2	Runbook de incidente — uma página	Recuperar rápido, por qualquer um	1 hora
3	Backup próprio + teste de restauração	Não ficar refém; dados na empresa	1 dia
4	Verificar PITR no painel (Database → Backups)	RPO de minutos, sem montar nada	15 min
5	Ambiente de homologação (PoC do stack próprio)	Testar antes do usuário + provar viabilidade de migrar	1 semana
6	Revisão trimestral de capacidade	Não repetir "meses em 1 GB sem perceber"	15 min/trimestre
7	Réplica e failover	Máquina morrer de vez	alto

Por que nesta ordem

Os itens 1 a 4 custam menos de uma semana somados e atacam exatamente o que machucou: ninguém soube da queda, e a recuperação dependeu de uma pessoa. O item 7 protege contra o único cenário que **não aconteceu nenhuma vez** — e por isso vem por último, não por ser ruim.

9. Armadilhas descobertas na prática

Erros reais cometidos durante a investigação, registrados para não se repetirem.

ARMADILHA	COMO EVITAR
<code>pg_stat_database.temp_files</code> não zera no restart . Lido logo após um reinício, parece atividade recente.	Para medir taxa, amostre duas vezes e tire a diferença. Nunca divida o acumulado pelo uptime.
<code>npm run build</code> (Vite/esbuild) não checa tipos . Um erro de tipo real passou por dois builds verdes.	Rodar <code>npx tsc --noEmit -p tsconfig.app.json</code> . Sem <code>-p</code> ele compila zero arquivos e sempre passa.
Disco cheio ≠ memória cheia . O disco estava em 47% nas três quedas.	Limpar tabela grande libera disco, não RAM. Não confundir os dois na hora do incidente.
Generalizar a partir de amostra . Duas tabelas grandes com zero leitura levaram à conclusão errada de que 80% do banco era descartável — o schema é usado pelo Espaço do Colaborador e pelo módulo contábil.	Verificar tabela por tabela em <code>pg_stat_all_tables</code> e no código antes de propor mover qualquer coisa.
Pool do PostgREST é fixo . Mantém as mesmas conexões com 1 consulta/s ou 70.	Reduzir consultas não reduz conexões. São problemas separados.

10. Anexo — baseline medida em produção

Números levantados em 21 e 23/09/2026, direto do banco. Servem de ponto de partida para a Fase 0 do plano de DevOps.

Banco por carga de trabalho

CARGA	TAMANHO	%	LIDO PELO ERP?
public — ERP transacional	239 MB	15%	sim
public — mz_* (contábil)	589 MB	37%	sim — DRE, balancete, fluxo de caixa
espelho (cópia do MySQL)	686 MB	43%	sim — ponto, organograma, cargos
auth / cron / storage	72 MB	5%	infraestrutura

Carga e superfície

Pico de conexões medido	61 (21/09, sobre limite de 57)
Pool do PostgREST	fixo: 21 no Micro
Requisições/dia no PostgREST (painel)	~6.600 · < 0,1 req/s média
Pontos de consulta no frontend	513 useQuery · 581 .from()
Escritas e invalidações	478 mutations · 586 invalidateQueries
Chamadas ao Storage	138
Tarefas agendadas (pg_cron)	5 · a mais frequente a cada 5 min, média 0,2 s
Testes automatizados	1.310

O número que muda o dimensionamento

Se as ~6.600 requisições/dia se confirmarem no levantamento de 7–14 dias, **20× isso ainda é cerca de 1,5 req/s**. Vale validar antes de dimensionar cluster, corretor de mensagens e cache — a diferença é de uma ordem de grandeza no custo.

Garantia transacional — verificada

Os caminhos de dinheiro do Malote passam por RPC, ou seja, executam como **transação única dentro do banco**:
malote_pagar_despesa , malote_pagar_parcela , malote_aprovar_despesa , malote_despesa_editar_pagamento — 21 operações no total.

Consequência prática: durante a queda de 23/09 os pagamentos **não foram salvos**, mas também **não foram salvos pela metade**. Nenhum registro ficou corrompido. O usuário viu erro e pôde refazer quando o sistema voltou.